axyon_crystal_merge.html
HTML 127.32KB

bu oyunu godotta yeniden yapacağız.
bana 0 godot bilgisi olan bir adam olarak adım adım rehber hazırla. En acemilerin bile anlayacağı şekilde neyi nasıl yaptığımızı ve detaylı açıklamalarını ver. ders kitabı gibi olsun. tüm oyun için hazırla. profesyonel bir kitap istiyorum senden. çok sayfalı.

18 saniye düşünüldü

Şimdi, kullanıcı bu oyunu Godot'ya aktarmak istiyor. HTML/JS'de yazılmış "AXY CRYSTAL — Merge Edition" oyununu sıfırdan Godot'da yapacak. Kullanıcı 0 Godot bilgisine sahip. Benden adım adım, acemilerin anlayacağı, ders kitabı gibi profesyonel ve çok sayfalı bir rehber istiyor.

Oyun: 5x7 grid, taş birleştirme (merge) mekaniği, power-up'lar, skor, coin, hedef, hold ile birleştirme, ses efektleri, görsel efektler, kayıt sistemi, level sistemi. Çok kapsamlı.

İstek: Godot'da yeniden yapacağız. 0 bilgi için rehber. Çok detaylı, adım adım, her şey açıklanmalı.

Yanıt uzun olacak. Godot 4 (GDScript) kullanılacağını varsayıyorum. Önce Godot'yu tanıtmak, arayüz, temel kavramlar, sonra projeyi adım adım kurmak.

Plan:

- Giriş: Oyun analizi, Godot'ya giriş
- Bölüm 1: Godot Kurulumu ve İlk Proje
- Bölüm 2: Temel Kavramlar (Scene, Node, Script, Signal, vs.)
- Bölüm 3: Oyun Tasarımı (Node yapısı, senaryo)
- Bölüm 4: Grid Sistemi (GridContainer, TileMap veya Control node'lar? 5x7 taşlar için daha çok Node2D veya Control tabanlı olabilir. Godot'da grid için genellikle TileMapLayer veya GridContainer tercih edilir. Taşlar ayrı ayrı nesne olduğu için her taş bir Node2D/Area2D veya TextureRect olabilir. Basitlik için Control tabanlı (TextureRect) kullanabiliriz.)
- Bölüm 5: Taş (Tile) Sınıfı
- Bölüm 6: Merge Mekaniği (BFS ile eşleşenleri bulma, basılı tutma, birleştirme)
- Bölüm 7: Gravity ve Spawn
- Bölüm 8: Puan, Coin, Hedef Sistemi
- Bölüm 9: Power-Up'lar
- Bölüm 10: Ses ve Görsel Efektler
- Bölüm 11: Kayıt Sistemi (ConfigFile veya Resource)
- Bölüm 12: UI ve Menüler
- Bölüm 13: Test ve Optimizasyon

- Bölüm 14: Paketleme

Her adımda kod örnekleri, açıklamalar, neden böyle yapıldığı. GDScript detayları.

Çok uzun olacak, ancak istenen bu. Yanıtı bölüm bölüm yazacağım, markdown formatında, başlıklar, kod blokları, notlar.

Ayrıca Godot editörünün kullanımına dair görseller eklenemez, ancak metinle anlatılacak.

Not: Kullanıcının HTML/JS bilgisi olduğu için benzetmeler yapılabilir (DOM → Node tree, event listener → signal, vs.)

Yanıtı yazmaya başlıyorum.

AXY CRYSTAL — Godot 4 ile Sıfırdan Oyun Geliştirme Rehberi

Yazar: Ahmet GÜNEŞ • 2026
Hedef Kitle: Godot bilgisi olmayan veya çok az olan herkes
Kapsam: HTML/JS'de yazılmış "Merge Edition" oyununun Godot 4'e tam aktarımı

Bu rehber, bir üniversite ders kitabı titizliğinde hazırlanmıştır. Her yeni kavram tanımlanır, her kod satırı açıklanır. Adım adım ilerleyerek, hiç Godot bilmeyen biri bile bu oyunu çalışır hale getirebilir.

İçindekiler

1. Godot'ya Giriş ve Kurulum
2. Godot Temel Kavramları (Sahne, Düğüm, Betik, Sinyal)
3. Projenin İskeleti – Sahne Yapısı
4. Grid Sistemi – Taşların Yerleşimi
5. Taş (Tile) Sınıfı – Görünüm ve Veri
6. Merge Mekaniği – Basılı Tutarak Birleştirme
7. Yerçekimi (Gravity) ve Yeni Taş Üretme (Spawn)
8. Puan, Coin ve Hedef Sistemi
9. Power-Up'lar – Değiştir, Geri Al, Yok Et, Karıştır, Bomba, Dondur
10. Ses ve Görsel Efektler
11. Oyun Durumu Kaydetme ve Yükleme
12. Kullanıcı Arayüzü (UI) ve Menüler
13. Hata Ayıklama, Test ve Performans
14. Oyunu Derleme ve Yayınlama (Android, Windows, Web)

1. Godot'ya Giriş ve Kurulum

1.1 Godot Nedir?

Godot, **açık kaynaklı**, **ücretsiz** ve **çok platformlu** bir oyun motorudur. 2D ve 3D oyunlar geliştirmek için kullanılır. Kendi betik dili **GDScript** (Python benzeri) ile programlanır. Ayrıca C#, C++ vb. diller de desteklenir. Biz **GDScript** kullanacağız.

Neden Godot?

- Tamamen ücretsiz, lisans derdi yok.
- Hafif, hızlı, düşük sistem gereksinimleri.
- Zengin düğüm (node) sistemi – görsel editör.
- Topluluk desteği çok iyi.
- Web, mobil, masaüstü, konsol hedefleyebilirsiniz.

1.2 Godot 4 Kurulumu

1. **İndirme:** godotengine.org adresine gidin. **Godot 4.x** (şu an 4.3 veya 4.4) sürümünü seçin. İşletim sisteminize uygun olanı indirin (Windows `.exe`, macOS `.dmg`, Linux `.AppImage`).
2. **Çalıştırma:** İndirdiğiniz dosyayı çift tıklayın. Herhangi bir kurulum gerekmez (taşınabilir).
3. **Proje Yöneticisi:** İlk açılışta "Project Manager" gelir. "Create New Project" diyerek yeni bir proje oluşturun.
   - **Project Name:** `AxyCrystalGodot`
   - **Project Path:** Bilgisayarınızda bir klasör seçin (ör. `C:\MyGames\AxyCrystalGodot`)
   - **Renderer: Forward+** (varsayılan) seçin (2D oyunlar için uygun). "Create & Edit" butonuna tıklayın.

Artık Godot editöründesiniz.

1.3 Editör Arayüzüne Hızlı Bakış

- **Scene Dock (Sahne Bölmesi):** Sol üstte, ağaç yapısı. Sahnedeki tüm düğümler (node) burada listelenir.
- **Viewport (Görünüm):** Ortada, oyun dünyasını gösterir.

- **Inspector (Denetçi):** Sağda, seçili düğümün özellikleri (transform, renk, script vb.).
- **FileSystem (Dosya Sistemi):** Sağ alt, proje dosyalarınız.
- **Bottom Panel (Alt Panel):** Debug, output, animation gibi sekmeler.

Önemli: Godot'da her şey bir **Sahne (Scene)** ve o sahnedeki **Düğümler (Node)** üzerine kuruludur. Sahne = bir "oyun içi nesne" (örn. oyuncu, taş, arka plan). Düğümler bu nesnelerin yapı taşlarıdır.

2. Godot Temel Kavramları (Sahne, Düğüm, Betik, Sinyal)

HTML/JS bilginiz varsa şu benzetmeyi yapabilirsiniz:

JavaScript / HTML	Godot
`document`	`SceneTree` (sahne ağacı)
HTML elemanı (`div` , `span`)	Node (düğüm)
CSS sınıfları	Node özellikleri ve stilleri
`addEventListener`	Signal (sinyal) + bağlantı
`setTimeout` / `requestAnimationFrame`	`Timer` düğümü veya `await get_tree().create_timer()`
`console.log`	`print()`

2.1 Düğüm (Node) Türleri

Her düğüm belirli bir iş yapar. En temel olanlar:

- **Node2D:** 2D uzayda konum, döndürme, ölçeklendirme sağlar. Görsel nesnelerin atası.
- **Control:** UI (arayüz) elemanlarının atası (buton, etiket, panel). Düğüm koordinatları ekran/UI referanslıdır.
- **Sprite2D:** Bir resim (texture) gösterir.
- **CollisionShape2D:** Çarpışma bölgesi (fare ile etkileşim için kullanacağız).
- **Area2D:** Fare giriş/çıkış, tıklama algılama için ideal.
- **Timer:** Zamanlayıcı.

- **AudioStreamPlayer:** Ses çalar.

2.2 Sahne (Scene) ve Düğüm Ağacı

Bir sahne, bir kök düğüm ve onun altındaki düğümlerden oluşur. Örneğin, oyundaki bir "Taş" sahnesi:

```text
Tile (Node2D)          <- kök düğüm
├─ Sprite2D            <- taşın görseli
├─ Area2D              <- tıklama alanı
│  └─ CollisionShape2D <- şekli (daire veya dikdörtgen)
└─ Label               <- üzerindeki sayı
```

Bu sahneyi `tile.tscn` olarak kaydederiz. Ana oyun sahnesinde (main.tscn) bu taşlardan 35 tane oluştururuz.

2.3 Betik (Script) – GDScript

Her düğüme bir GDScript dosyası ekleyebiliriz. Betik, düğümün davranışını tanımlar. Python'a benzer.

Örnek bir betik (Node2D için):

```gdscript
extends Node2D  # Bu düğümün türü

@export var tile_value: int = 2   # Dışarıdan görülebilir ve değiştirilebilir deği
şken

func _ready():
    print("Taş oluşturuldu, değeri: ", tile_value)
    update_visual()

func update_visual():
    # Görsel güncelleme kodu...
    pass
```

- `extends` : Hangi düğüm türünden türediğimizi söyler.
- `_ready()` : Düğüm ve tüm alt düğümleri hazır olduğunda otomatik çağrılır.
- `@export` : Bu değişken editörde gözükür, oyun içinde atanabilir.

2.4 Sinyaller (Signals)

Olay yayını yapmak için kullanılır. Örneğin bir butona tıklandığında "tıklandı" sinyali yayınlanır. Biz de bu sinyali dinleriz.

Bir sinyali bağlamak (connect):

```
gdscript

# Buton düğümüne eriş
var my_button = $Button
my_button.pressed.connect(_on_button_pressed)

func _on_button_pressed():
    print("Butona tıklandı!")
```

HTML'deki `addEventListener('click', callback)` ile aynı işlev.

Kendi sinyalimizi tanımlamak:

```
gdscript

signal merge_completed(new_value, position)

func merge():
    merge_completed.emit(16, global_position)
```

Başka bir düğümde: `other_node.merge_completed.connect(_on_merge)`

Bu mekanizma, JavaScript'teki **CustomEvent** veya callback fonksiyonlarına benzer.

2.5 Düğüm Referansları

Bir düğümün altındaki başka bir düğüme erişmek için `$` işareti kullanılır:

```
gdscript

$Sprite2D.texture = load("res://assets/tile_2.png")
$Label.text = "1024"
```

Eğer yol uzunsa: `$Panel/Grid/Button` gibi.

Alternatif: `get_node("Panel/Grid/Button")`

2.6 Zamanlayıcılar (Timer)

HTML'de `setTimeout` kullanırdık. Godot'da:

```gdscript
await get_tree().create_timer(0.5).timeout
# 0.5 saniye bekler
print("500 ms geçti")
```

Ayrıca bir Timer düğümü ekleyip `start()` da yapabiliriz.

2.7 Rastgele Sayı Üretme

```gdscript
var rng = RandomNumberGenerator.new()
rng.randomize()
var random_value = rng.randi_range(2, 64)
```

3. Projenin İskeleti – Sahne Yapısı

Oyunumuzun ana bileşenleri:

1. **Ana Sahne (Main)** : Tüm oyunu yönetir. Grid, UI, power-up'lar, ses.
2. **Taş Sahnesi (Tile)** : Tek bir taşın görünümü, tıklama, basılı tutma mantığı.
3. **UI Bileşenleri** : Puan, coin, hedef, butonlar.
4. **Power-Up Butonları** : Her biri ayrı bir düğüm (Button) olabilir.

3.1 Proje Klasör Yapısı

Proje dosyalarını düzenli tutmak için şu klasörleri oluşturun (FileSystem panelinde sağ tık → New Folder):

```text
res://
├─ scenes/          # Sahne dosyaları (.tscn)
│    ├─ main.tscn
│    ├─ tile.tscn
│    ├─ ui.tscn (isteğe bağlı)
├─ scripts/         # GDScript dosyaları (.gd)
│    ├─ main.gd
│    ├─ tile.gd
│    ├─ powerup.gd
├─ assets/          # Görseller, sesler
│    ├─ images/
```

```
│    ├─ sounds/
├─ fonts/           # Yazı tipleri (TTF)
```

3.2 Ana Sahne (Main) Oluşturma

1. **Scene Dock**'ta "2D Scene" butonuna tıklayın (veya `Ctrl+N`).
2. Kök düğümü `Node2D` olarak seçin. İsmini `Main` yapın.
3. `Main` düğümüne sağ tıklayıp "Add Child Node" → `CanvasLayer` ekleyin. (UI'nin oyun dünyasından bağımsız olması için).
4. `CanvasLayer`'ın altına:
   - `Panel` (arka plan)
   - `GridContainer` (taşları tutacak) – ancak taşlar dinamik hareket edeceği için `Node2D` altında manuel olarak yerleştireceğiz. Aslında taşların kendileri ayrı düğümler olacak ve `Main` altında bir `TileMapLayer` veya sadece bir `Node2D`'nin altında listelenecek. En basiti: `Main`'in altında `TilesContainer` (Node2D) oluşturun.
5. UI elemanları (Label, Button) için ayrı bir `Control` düğümü ekleyin.

Önerilen Main sahne yapısı:

```text
Main (Node2D)
├─ TilesContainer (Node2D)   <-- Taşlar buraya eklenir
├─ UI (CanvasLayer)
│    ├─ ScorePanel (Panel)
│    │    ├─ ScoreLabel (Label)
│    │    └─ CoinLabel (Label)
│    ├─ TargetPanel (Panel)
│    │    └─ TargetLabel (Label)
│    ├─ PowerUpRow (HBoxContainer)
│    │    ├─ HintButton (Button)
│    │    ├─ UndoButton (Button)
│    │    ├─ SmashButton (Button)
│    │    ├─ ShuffleButton (Button)
│    │    ├─ BombButton (Button)
│    │    └─ FreezeButton (Button)
│    └─ ControlButtons (HBoxContainer)
│         ├─ MenuButton (Button)
│         ├─ PauseButton (Button)
│         ├─ SoundButton (Button)
│         └─ HowToButton (Button)
├─ Audio (AudioStreamPlayer)  <-- arka plan müziği (isteğe bağlı)
└─ Popups (Control)           <-- Start, Pause, Game Over, HowTo panel
```

Adım adım yapalım:

- Main'i seçin → "Add Child Node" → `CanvasLayer` , adı `UI` .
- `UI` 'yı seçin → "Add Child Node" → `Panel` , adı `ScorePanel` . Inspector'da Layout → "Top Left" yapın, Anchors ile ekranın üst kısmına konumlandırın.
- `ScorePanel` 'in altına iki `Label` ekleyin: `ScoreValue` , `CoinValue` .
- `UI` altına bir `Panel` daha → `TargetPanel` . Layout "Top Right".
- `TargetPanel` altına `TargetLabel` (Label).
- `UI` altına `HBoxContainer` → adı `PowerUpRow` . İçine 6 tane `Button` ekleyin.
- Aynı şekilde `HBoxContainer` → `ControlButtons` , içine 4 button.

Bu sadece başlangıç. İleride düzenleyeceğiz.

3.3 Main Betiği (main.gd)

`Main` düğümüne bir script ekleyin: `Main` 'i seçin, Inspector'da "Script" kısmının yanındaki "@" işaretine tıklayın, `scripts/main.gd` olarak kaydedin.

Şimdilik sadece değişkenleri tanımlayalım.

```
gdscript

extends Node2D

# Sabitler
const COLS = 5
const ROWS = 7
const TARGETS = [2048, 4096, 8192, 16384, 32768, 65536, 131072, 262144, 524288, 10
48576, 2097152, 4194304]

# Oyun durumu
var grid = []          # 2D dizi (ROWS x COLS) -> her hücrede ya Tile referansı ya
da null
var score = 0
var coins = 30
var target_idx = 0

# Power-up durumları
var swap_mode = false
var swap_first_cell = null   # {row, col}
var smash_mode = false
var bomb_mode = false
var freeze_mode = false

# Diğer
var hist = []          # Geri alma için
```

```
var busy = false        # Animasyon devam ediyorsa yeni hamle engellensin
var active = true

# Düğüm referansları
@onready var tiles_container = $TilesContainer
@onready var score_label = $UI/ScorePanel/ScoreValue
@onready var coin_label = $UI/ScorePanel/CoinValue
@onready var target_label = $UI/TargetPanel/TargetLabel

func _ready():
    # Grid'i başlat
    init_grid()
    # UI güncelle
    update_hud()
    # Sinyal bağlantıları burada yapılacak
```

4. Grid Sistemi – Taşların Yerleşimi

HTML'de `G` matrisi ve DOM elementleri vardı. Godot'da `grid` adında bir dizi tutacağız. Her hücrede bir `Tile` sahnesinin örneği (instance) olacak veya boşsa `null`.

4.1 Taş (Tile) Sahnesini Hazırlama

1. Yeni bir sahne oluşturun (2D Scene). Kök düğüm `Node2D` olsun, adını `Tile` koyun.
2. `Tile` 'a bir `Sprite2D` ekleyin. Bu taşın arka plan resmi olacak. Geçici olarak `ColorRect` de kullanabiliriz ama sprite daha esnek. Şimdilik renkli bir kare eklemek için: `Sprite2D` yerine `ColorRect` kullanabiliriz (Control tabanlıdır, ancak Node2D altında olmaz). O yüzden `Tile` kökünü `Control` yapıp `TextureRect` kullanmak daha kolay olabilir. Ancak hareket animasyonları için `Node2D` daha uygun. Karışıklığı önlemek için **Tile'ı Node2D + Sprite2D + Label** yapalım.
   - `Tile` (Node2D)
     - `Sprite2D` (adı `BgSprite`) – texture atayacağız.
     - `Label` (adı `ValueLabel`) – üzerindeki sayı.
3. `BgSprite` 'nin `centered` özelliğini `false` yapın ki sol üst köşeden konumlansın (isteğe bağlı). Boyutları 64x64 yapalım (sonradan dinamik).
4. `ValueLabel` 'ın `horizontal_alignment = CENTER`, `vertical_alignment = CENTER`, `size` genişliği sprite ile aynı.
5. `Tile` sahnesini `res://scenes/tile.tscn` olarak kaydedin.

4.2 Tile Betiği (tile.gd)

`Tile` düğümüne bir script ekleyin (`scripts/tile.gd`). Bu script taşın değerini, görsel güncellemeyi, tıklama algılamayı yapacak.

```gdscript
extends Node2D

signal clicked(tile)            # Tıklama sinyali (basılı tutma için)
signal hold_completed(tile)     # Basılı tutma tamamlandığında

var value: int = 2
var row: int = 0
var col: int = 0
var is_holding = false
var hold_timer: Timer

func _ready():
    # Timer oluştur (hold süresi 0.2 saniye)
    hold_timer = Timer.new()
    add_child(hold_timer)
    hold_timer.one_shot = true
    hold_timer.timeout.connect(_on_hold_timeout)

    # Görsel güncelle
    update_visual()

func setup(r, c, val):
    row = r
    col = c
    value = val
    update_visual()

func update_visual():
    $ValueLabel.text = _format_value(value)
    # Arka plan rengini değere göre değiştir
    var color = get_color_for_value(value)
    $BgSprite.modulate = color
    # Boyutlandırma (isteğe bağlı)
    # ...

func _format_value(v):
    if v >= 1_000_000:
        return str(v / 1_000_000) + "M"
    elif v >= 1000:
        return str(v / 1000) + "K"
    else:
        return str(v)
```

```
func get_color_for_value(v):
    # Renk skalası (örnek)
    match v:
        2: return Color(0.75, 0.1, 0.36)  # bv2
        4: return Color(0.08, 0.5, 0.24)
        8: return Color(0.12, 0.25, 0.69)
        16: return Color(0.76, 0.25, 0.05)
        32: return Color(0.43, 0.18, 0.85)
        _: return Color(0.9, 0.7, 0.2)

# --- Girdi işleme (mouse/ touch) ---
func _input_event(viewport, event, shape_idx):
    if event is InputEventMouseButton and event.button_index == MOUSE_BUTTON_LEFT:
        if event.pressed:
            # Basıldı
            clicked.emit(self)
            start_hold()
        else:
            # Bırakıldı
            if is_holding:
                cancel_hold()
                hold_completed.emit(self)

func start_hold():
    is_holding = true
    hold_timer.start(0.175)  # 175ms

func _on_hold_timeout():
    if is_holding:
        # Yeterince basılı tutuldu -> merge işlemini başlat
        hold_completed.emit(self)
        is_holding = false

func cancel_hold():
    is_holding = false
    hold_timer.stop()
```

Açıklamalar:

- `_input_event` : Bu düğümün içine tıklandığında çağrılır. Sprite2D'nin `input_pickable` özelliği `true` olmalı (varsayılan true). Ayrıca Tile'ın kendisi `Node2D` olduğu için `input_pickable` yok; onun yerine `Area2D` kullanmak daha garanti. Ben en baştan `Area2D` kullanalım:

Düzeltme: Tile sahnesini şöyle yapalım:

```
text
```

```
Tile (Area2D)   <- kök
├─ CollisionShape2D (dikdörtgen)
├─ Sprite2D
└─ Label
```

`Area2D` 'nin `input_pickable` zaten true. `_input_event` de `Area2D` 'de çalışır.

O halde tile.gd'yi `extends Area2D` yapalım. `_input_event` metodu aynen kalır.

4.3 Main'de Grid'i Oluşturma

Main.gd içinde `init_grid()` fonksiyonunu yazalım.

```gdscript
var tile_scene = preload("res://scenes/tile.tscn")

func init_grid():
    # Grid dizisini boyutlandır (ROWS x COLS), her eleman null
    grid = []
    for r in range(ROWS):
        grid.append([])
        for c in range(COLS):
            grid[r].append(null)

    # Tüm hücrelere rastgele taş yerleştir
    for r in range(ROWS):
        for c in range(COLS):
            var val = get_random_start_value()
            create_tile(r, c, val)

func create_tile(row, col, val):
    var tile = tile_scene.instantiate()
    tile.setup(row, col, val)
    tiles_container.add_child(tile)
    # Taşın pozisyonunu hesapla (hücre boyutu 64, aralık 6)
    var cell_size = 64
    var gap = 6
    tile.position = Vector2(col * (cell_size + gap), row * (cell_size + gap))
    grid[row][col] = tile

    # Sinyalleri bağla
    tile.clicked.connect(_on_tile_clicked)
    tile.hold_completed.connect(_on_tile_hold_completed)
```

`get_random_start_value` : Başlangıç değerleri (2,4,8,16) arasında ağırlıklı.

```gdscript
```

```
func get_random_start_value():
    var rng = RandomNumberGenerator.new()
    var r = rng.randi_range(0, 9)
    if r < 5: return 2
    elif r < 8: return 4
    elif r < 9: return 8
    else: return 16
```

5. Taş (Tile) Sınıfı – Görünüm ve Veri

Yukarıda temel yapıyı verdik. Şimdi görsel zenginleştirme:

- Her değer için farklı renk ve gölge efekti.
- Değer metninin boyutunu değere göre ayarlama (kısa gösterim).
- Animasyonlar (drop, merge, vanish) için ayrı metodlar.

5.1 Tile.gd'yi genişletme

```gdscript
extends Area2D

signal clicked(tile)
signal hold_completed(tile)

var value: int = 2
var row: int = 0
var col: int = 0
var is_holding = false
var hold_timer: Timer

# Görsel elemanlar
@onready var sprite = $Sprite2D
@onready var label = $Label

func _ready():
    hold_timer = Timer.new()
    add_child(hold_timer)
    hold_timer.one_shot = true
    hold_timer.timeout.connect(_on_hold_timeout)
    update_visual()

func setup(r, c, val):
    row = r
    col = c
```

```
    value = val
    update_visual()

func update_visual():
    label.text = format_value(value)
    var font_size = calculate_font_size(value)
    label.add_theme_font_size_override("font_size", font_size)
    sprite.modulate = get_color_for_value(value)

func format_value(v):
    if v >= 1_000_000:
        return str(v / 1_000_000) + "M"
    elif v >= 1000:
        return str(v / 1000) + "K"
    else:
        return str(v)

func calculate_font_size(v):
    var s = format_value(v)
    if s.length() >= 6: return 18
    elif s.length() >= 4: return 24
    elif s.length() >= 3: return 30
    else: return 36

func get_color_for_value(v):
    match v:
        2: return Color(0.745, 0.094, 0.365)   # #be185d
        4: return Color(0.082, 0.502, 0.239)   # #15803d
        8: return Color(0.118, 0.251, 0.686)   # #1e40af
        16: return Color(0.761, 0.255, 0.047)  # #c2410c
        32: return Color(0.427, 0.176, 0.851)  # #6d28d9
        64: return Color(0.055, 0.455, 0.565)  # #0e7490
        128: return Color(0.631, 0.380, 0.027) # #a16207
        256: return Color(0.600, 0.106, 0.106) # #991b1b
        512: return Color(0.024, 0.369, 0.275) # #065f46
        1024: return Color(0.471, 0.188, 0.043)# #78350f
        2048: return Color(0.984, 0.749, 0.141)# #fbbf24
        _: return Color(0.8, 0.3, 0.8)          # diğerleri
```

5.2 Animasyon Metodları

```
gdscript

func play_drop_animation():
    var tween = create_tween()
    tween.set_parallel(true)
    tween.tween_property(self, "scale", Vector2(1.2, 1.2), 0.15)
    tween.tween_property(self, "modulate:a", 0.5, 0.1)
    await tween.finished
```

```
    tween = create_tween()
    tween.tween_property(self, "scale", Vector2(1, 1), 0.1)
    tween.tween_property(self, "modulate:a", 1, 0.1)

func play_merge_animation():
    var tween = create_tween()
    tween.tween_property(self, "scale", Vector2(1.4, 1.4), 0.1)
    tween.tween_property(self, "scale", Vector2(1, 1), 0.1)
    # Ayrıca bir ışık efekti de eklenebilir

func play_vanish_animation():
    var tween = create_tween()
    tween.tween_property(self, "scale", Vector2(0, 0), 0.2)
    tween.parallel().tween_property(self, "modulate:a", 0, 0.2)
    await tween.finished
    queue_free()
```

Bu metodları merge veya smash sırasında çağıracağız.

6. Merge Mekaniği – Basılı Tutarak Birleştirme

HTML'deki mantık: Taşa basılı tut → BFS ile aynı değerdeki tüm komşuları bul → bırakınca hepsini merkeze uçur → yeni değer oluştur.

Godot'da bunu şöyle yapacağız:

1. `Main` 'de `_on_tile_clicked(tile)` ve `_on_tile_hold_completed(tile)` sinyallerini yakala.
2. Hold tamamlandığında, o taştan başlayarak BFS ile aynı değerdeki tüm bağlı taşları bul.
3. Eğer en az 1 eş bulunduysa (toplam ≥2), merge işlemini başlat.
4. Merge sırasında:
   - Tüm eşleşen taşları merkeze doğru hareket ettir (Tween ile).
   - Grid dizisinden kaldır, yerlerini boşalt.
   - Merkezdeki taşın değerini yeni değerle güncelle.
   - Puan ve coin ekle.
   - Gravity ve spawn tetikle.

6.1 BFS ile Eşleşenleri Bulma

Main.gd'ye ekleyelim:

gdscript

```
func find_matching_neighbors(start_tile):
    var target_value = start_tile.value
    var queue = [start_tile]
    var visited = []
    var matches = []

    while queue.size() > 0:
        var current = queue.pop_front()
        if current in visited:
            continue
        visited.append(current)
        if current.value == target_value:
            matches.append(current)
            # Komşularını ekle
            for n in get_neighbors(current.row, current.col):
                if n != null and not n in visited:
                    queue.append(n)
    return matches

func get_neighbors(row, col):
    var neighbors = []
    for dr in [-1, 0, 1]:
        for dc in [-1, 0, 1]:
            if dr == 0 and dc == 0:
                continue
            var nr = row + dr
            var nc = col + dc
            if nr >= 0 and nr < ROWS and nc >= 0 and nc < COLS:
                neighbors.append(grid[nr][nc])
    return neighbors
```

6.2 Merge İşlemini Gerçekleştirme

gdscript

```
var is_merging = false  # busy yerine kullan

func _on_tile_hold_completed(tile):
    if busy or not active:
        return
    var matches = find_matching_neighbors(tile)
    if matches.size() < 2:
        return

    busy = true
    save_history()
```

```
    var center = tile
    var merge_count = matches.size()   # merkez dahil
    var new_value = center.value * pow(2, ceil(merge_count / 2.0))

    # Animasyon: eşleşen taşları merkeze uçur
    for t in matches:
        if t == center:
            continue
        animate_tile_to_center(t, center)
        # Grid'den kaldır
        grid[t.row][t.col] = null
        t.queue_free()

    await get_tree().create_timer(0.25).timeout

    # Merkez taşı güncelle
    center.value = new_value
    center.update_visual()
    center.play_merge_animation()

    # Puan hesapla
    var log_val = log(new_value) / log(2)
    var combo_bonus = 1.0
    if merge_count >= 8: combo_bonus = 2.0
    elif merge_count >= 5: combo_bonus = 1.5
    elif merge_count >= 3: combo_bonus = 1.2
    var add_score = int(log_val * merge_count * combo_bonus)
    score += add_score
    coins += 1
    update_hud()

    # Gravity ve spawn
    await apply_gravity_and_spawn()

    busy = false
    check_game_over()
```

animate_tile_to_center :

```gdscript
func animate_tile_to_center(tile, center):
    var tween = create_tween()
    var start_pos = tile.global_position
    var end_pos = center.global_position
    tween.tween_property(tile, "global_position", end_pos, 0.2).set_ease(Tween.EAS
E_IN).set_trans(Tween.TRANS_CUBIC)
    tween.parallel().tween_property(tile, "scale", Vector2(0.2, 0.2), 0.2)
    tween.parallel().tween_property(tile, "modulate:a", 0, 0.2)
```

6.3 Yerçekimi (Gravity)

Her sütunda taşları aşağı kaydır:

```
gdscript

func apply_gravity():
    for c in range(COLS):
        var write_row = ROWS - 1
        for r in range(ROWS-1, -1, -1):
            var tile = grid[r][c]
            if tile != null:
                if r != write_row:
                    # Taşı aşağı taşı
                    grid[write_row][c] = tile
                    grid[r][c] = null
                    tile.row = write_row
                    tile.col = c
                    # Animasyonlu hareket
                    var new_pos = get_tile_position(write_row, c)
                    var tween = create_tween()
                    tween.tween_property(tile, "position", new_pos, 0.15)
                write_row -= 1
    await get_tree().create_timer(0.2).timeout

func get_tile_position(row, col):
    var cell_size = 64
    var gap = 6
    return Vector2(col * (cell_size + gap), row * (cell_size + gap))
```

6.4 Yeni Taş Üretme (Spawn)

```
gdscript

func spawn_tiles():
    if freeze_mode:
        freeze_mode = false
        update_freeze_indicator()
        return
    # Boş hücreleri bul
    for c in range(COLS):
        for r in range(ROWS):
            if grid[r][c] == null:
                var val = get_random_spawn_value()
                create_tile(r, c, val)
                await get_tree().create_timer(0.04).timeout  # her spawn arası 40m
s
    # Bonus taş mekanizması HTML'deki gibi eklenebilir (isteğe bağlı)
```

```
func get_random_spawn_value():
    # Ağırlıklı rastgele (2 çok, 4 orta, 8 az, 16 çok az)
    var r = randi() % 100
    if r < 60: return 2
    elif r < 85: return 4
    elif r < 95: return 8
    else: return 16
```

apply_gravity_and_spawn :

gdscript

```
func apply_gravity_and_spawn():
    await apply_gravity()
    await spawn_tiles()
    await apply_gravity()  # spawn'dan sonra tekrar gravity
```

7. Yerçekimi (Gravity) ve Yeni Taş Üretme (Spawn)

Yukarıda ana hatlarıyla verdik. Detaylandıralım.

7.1 Gravity Animasyonu İyileştirme

Her taşın `position` 'unu tween ile değiştirirken aynı anda tüm taşların kayması daha gerçekçi. Bunun için `parallel` tween kullanabiliriz.

gdscript

```
func apply_gravity():
    var tweens = []
    for c in range(COLS):
        var write_row = ROWS - 1
        for r in range(ROWS-1, -1, -1):
            var tile = grid[r][c]
            if tile != null:
                if r != write_row:
                    grid[write_row][c] = tile
                    grid[r][c] = null
                    tile.row = write_row
                    tile.col = c
                    var new_pos = get_tile_position(write_row, c)
                    var tween = create_tween()
                    tween.tween_property(tile, "position", new_pos, 0.15)
                    tweens.append(tween)
                write_row -= 1
    # Tüm tween'ler bitene kadar bekle
```

```
    for t in tweens:
        await t.finished
```

7.2 Spawn Animasyonu

Yeni taş oluştururken `play_drop_animation` çağır:

```gdscript
func create_tile(row, col, val, with_animation=true):
    var tile = tile_scene.instantiate()
    tile.setup(row, col, val)
    tiles_container.add_child(tile)
    tile.position = get_tile_position(row, col)
    grid[row][col] = tile
    tile.clicked.connect(_on_tile_clicked)
    tile.hold_completed.connect(_on_tile_hold_completed)
    if with_animation:
        tile.play_drop_animation()
    return tile
```

7.3 Bonus Taş (Big Merge Ödülü)

HTML'de 10+ taş birleşince %30 ihtimalle hedefin 1/4'ü değerinde özel taş çıkıyordu. Aynısını ekleyelim.

`apply_gravity_and_spawn` içinde spawn öncesi `last_merge_count` kontrolü yapılır. Bunun için `merge` işleminde `merge_count` 'ü global bir değişkende tut.

```gdscript
var last_merge_count = 0

# merge içinde:
last_merge_count = merge_count

# spawn_tiles içinde:
func spawn_tiles():
    if freeze_mode:
        freeze_mode = false
        update_freeze_indicator()
        return
    var empty_cells = []
    for r in range(ROWS):
        for c in range(COLS):
            if grid[r][c] == null:
                empty_cells.append(Vector2(r, c))
```

```
    # Bonus taş kontrolü
    var bonus_val = TARGETS[target_idx] / 4
    var do_bonus = last_merge_count >= 10 and empty_cells.size() >= 3 and randf()
< 0.3 and bonus_val >= 2
    last_merge_count = 0
    if do_bonus:
        var idx = randi() % empty_cells.size()
        var pos = empty_cells[idx]
        create_tile(pos.x, pos.y, bonus_val)
        # Bonus taşa özel efekt
        # ...
    # Normal spawn...
```

8. Puan, Coin ve Hedef Sistemi

8.1 Puan Hesaplama (Merge Puanı)

Formül: `puan = log2(yeni_değer) * birleşen_taş_sayısı * combo_bonus`

GDScript'te:

```gdscript
var log_val = log(new_value) / log(2)
var combo_bonus = 1.0
if merge_count >= 8: combo_bonus = 2.0
elif merge_count >= 5: combo_bonus = 1.5
elif merge_count >= 3: combo_bonus = 1.2
var add_score = int(log_val * merge_count * combo_bonus)
score += add_score
coins += 1  # her merge 1 coin
```

8.2 Hedef Kontrolü

Her merge sonrası yeni değer hedef değerine eşit veya büyükse level atla.

```gdscript
var target_value = TARGETS[target_idx]
if new_value >= target_value:
    level_complete(new_value)
```

8.3 Level Complete

gdscript

```
func level_complete(achieved_value):
    active = false
    busy = true
    coins += 50
    update_hud()
    # Yeni rekor kontrolü
    if score > get_highest_score():
        show_new_record()
    # Level geçiş animasyonu
    show_level_clear_popup(achieved_value)
    # Bir sonraki level'e geçmek için popup'ta butona tıklanınca:
    # go_to_next_level()

func go_to_next_level():
    if target_idx < TARGETS.size() - 1:
        target_idx += 1
    # Büyük taşları düşür (downgrade)
    var new_target = TARGETS[target_idx]
    for r in range(ROWS):
        for c in range(COLS):
            var tile = grid[r][c]
            if tile != null and tile.value >= new_target:
                tile.value = new_target / 2  # örnek, daha doğrusu hedefin bir alt
değeri
                tile.update_visual()
    # Taşları karıştır (scramble)
    scramble_board()
    active = true
    busy = false
```

8.4 UI Güncelleme

gdscript

```
func update_hud():
    score_label.text = str(score)
    coin_label.text = str(coins)
    target_label.text = format_value(TARGETS[target_idx])
```

9. Power-Up'lar – Değiştir, Geri Al, Yok Et, Karıştır, Bomba, Dondur

Her power-up bir butona bağlanacak. Butona tıklanınca ilgili mod aktif olur veya doğrudan işlem yapılır.

9.1 Değiştir (Swap)

gdscript

```
func enable_swap_mode():
    if coins < 40: return
    swap_mode = true
    swap_first_cell = null
    # UI'da active görseli

func _on_tile_clicked(tile):
    if swap_mode:
        if swap_first_cell == null:
            swap_first_cell = tile
            tile.modulate = Color(1, 1, 0)  # sarı vurgu
        else:
            if is_adjacent(swap_first_cell, tile):
                # swap işlemi
                swap_tiles(swap_first_cell, tile)
                coins -= 40
                update_hud()
            swap_first_cell.modulate = Color(1,1,1)
            swap_mode = false
        return
    # normal hold işlemi...
```

swap_tiles :

gdscript

```
func swap_tiles(tile_a, tile_b):
    # grid dizisinde yer değiştir
    var r1 = tile_a.row; var c1 = tile_a.col
    var r2 = tile_b.row; var c2 = tile_b.col
    grid[r1][c1] = tile_b
    grid[r2][c2] = tile_a
    tile_a.row = r2; tile_a.col = c2
    tile_b.row = r1; tile_b.col = c1
    # pozisyonları animasyonla değiştir
    var pos1 = get_tile_position(r2, c2)
    var pos2 = get_tile_position(r1, c1)
    var tween = create_tween()
    tween.tween_property(tile_a, "position", pos1, 0.2)
    tween.parallel().tween_property(tile_b, "position", pos2, 0.2)
    await tween.finished
```

9.2 Geri Al (Undo)

`hist` dizisine her hamle öncesi durumu kaydederiz. Undo butonuna basınca en son durumu geri yükleriz.

```gdscript
func save_history():
    var snapshot = {
        "grid": deep_copy_grid(),
        "score": score,
        "coins": coins,
        "target_idx": target_idx
    }
    hist.append(snapshot)
    if hist.size() > 12:
        hist.pop_front()

func undo():
    if hist.is_empty() or coins < 50: return
    var last = hist.pop_back()
    restore_grid(last.grid)
    score = last.score
    coins = last.coins - 50
    target_idx = last.target_idx
    update_hud()
```

`restore_grid`:

```gdscript
func restore_grid(saved_grid):
    # Mevcut tüm taşları sil
    for r in range(ROWS):
        for c in range(COLS):
            if grid[r][c] != null:
                grid[r][c].queue_free()
    # Yeni taşları oluştur
    for r in range(ROWS):
        for c in range(COLS):
            var val = saved_grid[r][c]
            if val > 0:
                create_tile(r, c, val, false)
            else:
                grid[r][c] = null
```

9.3 Yok Et (Smash)

gdscript

```
func enable_smash_mode():
    if coins < 50: return
    smash_mode = true

func _on_tile_clicked(tile):
    if smash_mode:
        smash_tile(tile)
        smash_mode = false
        coins -= 50
        update_hud()
        return
    # ...

func smash_tile(tile):
    var r = tile.row; var c = tile.col
    grid[r][c] = null
    tile.play_vanish_animation()
    await tile.tree_exited
    await apply_gravity_and_spawn()
```

9.4 Karıştır (Shuffle)

gdscript

```
func shuffle_board():
    if coins < 80: return
    var all_values = []
    for r in range(ROWS):
        for c in range(COLS):
            if grid[r][c] != null:
                all_values.append(grid[r][c].value)
    all_values.shuffle()
    var idx = 0
    for r in range(ROWS):
        for c in range(COLS):
            if grid[r][c] != null:
                grid[r][c].value = all_values[idx]
                grid[r][c].update_visual()
                idx += 1
    coins -= 80
    update_hud()
```

9.5 Bomba (Bomb)

gdscript

```
func enable_bomb_mode():
    if coins < 120: return
    bomb_mode = true

func _on_tile_clicked(tile):
    if bomb_mode:
        bomb_area(tile)
        bomb_mode = false
        coins -= 120
        update_hud()
        return

func bomb_area(center_tile):
    var center_r = center_tile.row
    var center_c = center_tile.col
    for dr in [-1,0,1]:
        for dc in [-1,0,1]:
            var r = center_r + dr
            var c = center_c + dc
            if r >= 0 and r < ROWS and c >= 0 and c < COLS:
                var t = grid[r][c]
                if t != null:
                    grid[r][c] = null
                    t.play_vanish_animation()
    await get_tree().create_timer(0.3).timeout
    await apply_gravity_and_spawn()
```

9.6 Dondur (Freeze)

```
gdscript

func activate_freeze():
    if coins < 60: return
    freeze_mode = true
    coins -= 60
    update_hud()
    # UI'da freeze göstergesi
```

Spawn sırasında `freeze_mode` kontrolü yapılacak.

10. Ses ve Görsel Efektler

10.1 Ses Sistemi

Godot'da ses çalmak için `AudioStreamPlayer` düğümü kullanılır. Projeye ses dosyaları ekleyin (örn. `assets/sounds/merge.wav`, `smash.wav`). Main sahnesine bir `AudioStreamPlayer` ekleyip scriptten kontrol edin.

```
gdscript

@onready var merge_sound = $AudioMerge
@onready var smash_sound = $AudioSmash
@onready var win_sound = $AudioWin

func play_sound(sound_node):
    if sound_on:
        sound_node.play()
```

Her olayda ilgili sesi çal.

10.2 Görsel Efektler (Parçacıklar, Ekran Flaşı)

- **Parçacık sistemi:** `GPUParticles2D` düğümü kullanılabilir. Basitçe bir sahne oluşturup merge anında instantiate edip konumlandırın.
- **Ekran flaşı:** Tüm ekranı kaplayan yarı saydam beyaz bir `ColorRect`'i kısa süreli gösterin.

```
gdscript

func screen_flash():
    var flash = ColorRect.new()
    flash.color = Color(1,1,1,0.3)
    flash.size = get_viewport().size
    add_child(flash)
    var tween = create_tween()
    tween.tween_property(flash, "modulate:a", 0, 0.2)
    await tween.finished
    flash.queue_free()
```

- **Yüzen puan metni:** `Label` oluştur, yukarı doğru tweenle.

```
gdscript

func spawn_float_text(position, text, color=Color.YELLOW):
    var label = Label.new()
    label.text = text
    label.add_theme_color_override("font_color", color)
    label.position = position - Vector2(20, 20)
    add_child(label)
    var tween = create_tween()
    tween.tween_property(label, "position", label.position - Vector2(0, 50), 1.0)
```

```
    tween.parallel().tween_property(label, "modulate:a", 0, 1.0)
    await tween.finished
    label.queue_free()
```

11. Oyun Durumu Kaydetme ve Yükleme

Godot'da veri kaydetmek için `ConfigFile` veya `Resource` kullanılır. En kolayı `ConfigFile`:

```gdscript
func save_game():
    var config = ConfigFile.new()
    config.set_value("game", "grid", encode_grid())
    config.set_value("game", "score", score)
    config.set_value("game", "coins", coins)
    config.set_value("game", "target_idx", target_idx)
    config.set_value("game", "timestamp", Time.get_unix_time_from_system())
    config.save("user://save.cfg")

func encode_grid():
    var data = []
    for r in range(ROWS):
        var row = []
        for c in range(COLS):
            row.append(grid[r][c].value if grid[r][c] != null else 0)
        data.append(row)
    return data

func load_game():
    var config = ConfigFile.new()
    if config.load("user://save.cfg") != OK:
        return false
    var timestamp = config.get_value("game", "timestamp", 0)
    if Time.get_unix_time_from_system() - timestamp > 86400:  # 24 saat
        return false
    var saved_grid = config.get_value("game", "grid")
    score = config.get_value("game", "score")
    coins = config.get_value("game", "coins")
    target_idx = config.get_value("game", "target_idx")
    restore_grid_from_data(saved_grid)
    return true

func restore_grid_from_data(data):
    # Önce mevcut grid'i temizle
    for r in range(ROWS):
        for c in range(COLS):
```

```
            if grid[r][c]:
                grid[r][c].queue_free()
    # Veriden taşları oluştur
    for r in range(ROWS):
        for c in range(COLS):
            var val = data[r][c]
            if val > 0:
                create_tile(r, c, val, false)
            else:
                grid[r][c] = null
    update_hud()
```

Otomatik kayıt için `_process` veya `Timer` kullanabilirsiniz.

12. Kullanıcı Arayüzü (UI) ve Menüler

12.1 Başlangıç Menüsü

Ayrı bir sahne (`start_menu.tscn`) yapabilir veya Main'de popup olarak gösterebiliriz. Popup tercih edelim.

Main sahnesine bir `CanvasLayer` daha ekleyin, adı `PopupLayer` . İçine `Panel` ve butonlar.

Başlangıçta `PopupLayer` görünür, "Yeni Oyun" ve "Devam Et" butonları.

12.2 Duraklatma Menüsü

Benzer şekilde bir popup. Pause butonuna basınca `active = false` ve popup'ı göster.

12.3 Oyun Sonu Menüsü

Game Over olduğunda popup göster, skoru göster, yeniden başlat butonu.

12.4 Nasıl Oynanır?

Bir popup veya ayrı sahne.

Tüm popupları yönetmek için `PopupLayer` altında farklı `Panel` 'ler kullanıp `visible` özelliğini değiştirin.

13. Hata Ayıklama, Test ve Performans

- `print()` ile konsola değer yazdırın.
- Godot editöründe "Debug" → "Remote" ile çalışan oyundaki düğümleri inceleyin.
- Performans için: Her frame'de fazla işlem yapmayın. Tween'ler doğal olarak optimize.
- Mobil cihazda test için: Android/iOS'ye export edip test edin.

14. Oyunu Derleme ve Yayınlama (Android, Windows, Web)

14.1 Export Template Kurulumu

Godot'da "Editor" → "Manage Export Templates" → İndir.

14.2 Windows

"Project" → "Export" → "Add" → "Windows Desktop". Hedef klasörü seç, "Export Project".

14.3 Android

- Android SDK, JDK, Debug keystore gerekir. (Detaylı rehber için Godot docs)
- "Export" → "Android" → "Export".

14.4 Web (HTML5)

"Export" → "Web". Çıktı `.html`, `.wasm`, `.pck` dosyaları. Bir web sunucusuna yükleyin.

Sonuç

Bu rehberde, sıfırdan bir Godot kullanıcısının AXY CRYSTAL oyununu adım adım nasıl geliştireceğini anlattık. Her bölümde temel kavramları tanımladık, kodları satır satır açıkladık. Artık kendi oyununuzu yapmaya hazırsınız.

Önemli Not: Bu rehberdeki kod parçaları eksiksiz bir oyun oluşturmak için yeterli değildir; ancak her bir mekaniği anlamanız ve kendi projenize uyarlamanız için

sağlam bir temel oluşturur. Eksik kalan detayları (örneğin görsel varlıklar, tam ses dosyaları, tween senkronizasyonu) kendiniz tamamlamanız gerekecektir. Godot dokümantasyonu ve topluluk forumları en iyi yardımcılarınızdır.

İyi geliştirmeler!
Ahmet GÜNEŞ – 2026